

# Rapport technique

## 1. Introduction

Ce projet implémente un système de stockage et de diffusion d'événements horodatés (DateTimeEventStore).

L'objectif principal était de proposer une architecture extensible permettant :

- le stockage d'événements selon différentes stratégies ;
- l'ajout de mécanismes de notification via différents handlers ;
- une utilisation simple à travers une interface en ligne de commande (CLI).

Le projet a été développé en Python et s'appuie sur plusieurs patrons de conception afin de faciliter son évolution future. J'ai volontairement extrapolé de manière à considérer que la classe event store va être utilisé dans un environnement plutôt complexe style micro service. Si cette classe avait pour vocation d'être utilisée dans un petit script, ma solution est très "over engineered".

---

## 2. Architecture générale

L'application est organisée autour de trois concepts principaux :

### Event Store

Le composant principal est responsable du stockage et de la récupération des événements.

Un événement est actuellement composé :

- d'une date/heure ;
- d'une donnée sous forme de chaîne de caractères.

Cependant, l'architecture ne limite pas le contenu à une simple chaîne. Il serait possible de stocker n'importe quel document JSON sans modification majeure du design.

### Storage Strategy

Le stockage réel est délégué à une stratégie interchangeable.

Deux implémentations sont actuellement disponibles :

- stockage en mémoire ;
- stockage MongoDB.

### Event Handlers

Après l'ajout d'un événement, celui-ci peut être transmis à une liste de handlers responsables d'effectuer des actions complémentaires (affichage console, envoi RabbitMQ, requête HTTP, etc.).

---

## 3. Patrons de conception utilisés

### Factory Pattern

Une factory est utilisée pour créer les instances de DateTimeEventStore.

Cette approche permet d'isoler la logique de création des objets et de simplifier l'ajout de nouvelles implémentations à l'avenir.

### Strategy Pattern

Le stockage est abstrait derrière une interface commune.

Les stratégies actuellement disponibles sont :

- MongoDB ;
- stockage mémoire.

Cette abstraction permet de changer de backend sans impacter le reste de l'application.

Le stockage mémoire repose sur une SortedList contenant des tuples.

Ce choix a été effectué car :

- l'insertion s'effectue en  $O(\log n)$  ;
- la recherche reste efficace grâce au maintien permanent de l'ordre chronologique ;
- la date est utilisée comme clé de tri naturelle.

Cette architecture permettrait également d'ajouter facilement d'autres stratégies telles que :

- SQL ;
- Elasticsearch ;
- Redis ;
- tout autre système de persistance.

### Observer Pattern

Les handlers sont implémentés selon le patron Observer.

Le store notifie automatiquement les handlers enregistrés lors de l'ajout d'un nouvel événement.

Les handlers actuellement implémentés sont :

- ConsoleEventHandler ;
- RabbitMQEventHandler ;
- HttpEventHandler.

Le handler console a été développé initialement afin de faciliter les tests et le débogage.

Les handlers RabbitMQ et HTTP ont ensuite été ajoutés pour illustrer des cas d'utilisation plus réalistes.

À l'heure actuelle, le CLI ne permet d'enregistrer qu'une seule instance par type de handler.

Cette limitation provient uniquement de l'interface utilisateur et non de l'architecture. Le modèle interne permettrait sans difficulté d'avoir plusieurs handlers du même type avec des configurations différentes.

---

51.68.35.140:4215 > event\_store > events

Documents 116 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#) ✨

[ADD DATA](#) [UPDATE](#) [DELETE](#) [EXPORT DATA](#) [EXPORT CODE](#)

```
_id: ObjectId('6a2b5184b2ed33ac6ae1ab55')
date: "2026-06-12"
data: Object
```

```
_id: ObjectId('6a2bb4abc427de05b2b55078')
date: 1781249195.697519
data: "Event 0"
```

```
_id: ObjectId('6a2bb4abc427de05b2b55079')
date: 1781249195.781477
data: "Event 1"
```

```
_id: ObjectId('6a2bb4abc427de05b2b5507a')
date: 1781249195.788427
data: "Event 2"
```

```
_id: ObjectId('6a2bb4abc427de05b2b5507b')
date: 1781249195.797932
data: "Event 3"
```

Capture d'écran de la collection d'événement sur MongoDB Compass



RabbitMQ 3.13.7

Erlang 26.2.5.16

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Messages: 1

Get Message(s)

Message 1

The server reported 9 messages remaining.

|                              |                                                  |
|------------------------------|--------------------------------------------------|
| Exchange                     | (AMQP default)                                   |
| Routing Key                  | test-queue                                       |
| Redelivered                  | 0                                                |
| Properties                   | delivery_mode: 2                                 |
| Payload                      | { "date": 1781250963.228884, "data": "Event 0" } |
| 46 bytes<br>Encoding: string |                                                  |

Capture d'écran du message de la queue RabbitMq

## 4. Choix de conception écartés

J'ai envisagé l'utilisation d'une Abstract Factory afin d'abstraire davantage le concept de `DateTimeEventStore`.

Cette approche aurait par exemple permis de créer ultérieurement d'autres familles de stores spécialisées telles que :

- `DateTimeEventStore` ;
- `DateGeoSpatialEventStore` pour les événements physique;
- etc.

J'ai finalement décidé de ne pas retenir cette solution.

Bien qu'intéressante sur le plan théorique, elle introduit un niveau d'abstraction supplémentaire que j'ai jugé comme étant overkill.

J'ai privilégié une architecture plus simple respectant le principe YAGNI ("You Aren't Gonna Need It").

---

## 5. Tests

Les tests ont été réalisés avec pytest.

Les scénarios couverts portent principalement sur :

- les différentes stratégies de stockage ;
- les handlers d'événements.

J'ai volontairement conservé une suite de tests relativement légère afin de concentrer l'effort sur les fonctionnalités principales.

Il aurait également été possible de tester le CLI de manière plus exhaustive si cela avait été un objectif explicite du sujet.

L'utilisation d'outils d'assistance par IA a été utilisée pour accélérer la rédaction et la mise en place de certains tests.

---

## 6. Logging

L'application utilise le module logging standard de Python.

Actuellement le logging est au niveau WARNING pour plus de verbosité passer au niveau INFO

Les principaux événements journalisés sont :

- les opérations de stockage ;
- les appels aux handlers ;
- l'établissement des connexions MongoDB ;
- l'établissement des connexions RabbitMQ.

Pour un projet de plus grande taille ou destiné à la production, il serait probablement pertinent de mettre en place une couche de logging plus structurée et centralisée.

L'assistance de l'IA a également été utilisée pour cette partie.

---



## 7. Interface CLI

Une interface en ligne de commande a été développée à l'aide de Typer.

Elle permet notamment :

- l'ajout d'événements ;
- la consultation des événements ;
- l'export JSON ;
- l'utilisation d'une session interactive.

L'export JSON ainsi que le mode interactif ont été ajoutés avec l'aide d'outils d'assistance IA.

---

## 8. Temps passé

Temps total estimé : environ 10 heures.

Répartition approximative :

- révision des patrons de conception ;
- mise en place de MongoDB ;
- mise en place de RabbitMQ ;
- développement principal ;
- débogage et ajustements ;
- rédaction des tests.

Note : Le test a été envoyé le 11 juin à 15h pour une livraison le 12 juin à midi

---

## 9. Exécution du projet

Repository GitHub :

<https://github.com/cherfaoui-syphax/fastapi-test>

### Installation

Créer un environnement virtuel :

```
python -m venv myenv  
myenv\Scripts\activate
```

Installer les dépendances :

```
pip install -r requirements.txt
```

Copier le fichier `.env` fourni dans l'email à la racine du projet.

### Lancement

Mode interactif :

```
python main.py interactive
```

Les commandes peuvent également être exécutées individuellement mais le mode interactif est beaucoup plus pratique.

### Données de démonstration

La base MongoDB contient déjà plusieurs événements insérés pendant le développement.

Ils peuvent être exportés au format JSON afin de visualiser les données stockées.

---

## 10. Améliorations possibles

Plusieurs évolutions pourraient être envisagées :

- Sécuriser la base de données ainsi que l'instance rabbitMq derrière un pare-feu et un proxy nGinx. Pour me simplifier la vie l'architecture est actuellement très peu sécurisé
- Compléter le typage fait légèrement à la va vite.
- ajout d'opérations asynchrones pour le stockage et les handlers.
- Gérer le logging via un fichier de configuration
- import/reconstruction d'un store à partir d'un export JSON .
- Ajouter d'autre type d'export
- ajout de nouvelles stratégies de stockage.
- ajout de nouveaux handlers.
- amélioration de la couverture de tests.
- prise en charge de multiples handlers d'un même type.
- ajout de mécanismes de retry et de gestion avancée des erreurs pour RabbitMQ et HTTP.